

Efialtis Stin Kouzina

Food Hazard Detection - SemEval-2025 Task 9, Subtask 1

AM: 4940 ΦΟΥΡΚΙΩΤΗΣ ΑΘΑΝΑΣΙΟΣ

Εισαγωγή

Σε αυτή την εργασία ασχολήθηκα με την ταξινόμηση κειμένων που έχουν σχέση με ανακλήσεις τροφίμων. Κάθε εγγραφή είναι μια μικρή ανακοίνωση για κάποιο τρόφιμο που έχει πρόβλημα. Από αυτό το κείμενο θέλουμε να βρούμε δύο πράγματα: πρώτα τι είδους κίνδυνος υπάρχει και μετά σε ποια κατηγορία ανήκει το προϊόν.

Η λύση ονομάστηκε Efialtis Stin Kouzina (Εφιάλτης στην Κουζίνα). Το όνομα περιγράφει τι κάνει το σύστημα: εντοπίζει τους «εφιάλτες» που κρύβονται στις ανακλήσεις τροφίμων (αλλεργιογόνα, μολύνσεις, ξένα σώματα) πριν φτάσουν στο πιάτο. Η μέθοδος δουλεύει σε δύο βήματα: πρώτα προσπαθώ να εντοπίσω τον κίνδυνο, δηλαδή το hazard, και μετά χρησιμοποιώ αυτή την πρόβλεψη για να βοηθήσω την πρόβλεψη του product. Με απλά λόγια, το πρώτο βήμα δίνει μια έξτρα βοήθεια στο δεύτερο.

1. Τι ζητάει το πρόβλημα

Το task είναι το SemEval-2025 Task 9, Subtask 1. Είναι ένα supervised text classification πρόβλημα. Δηλαδή έχουμε παραδείγματα με σωστές απαντήσεις και εκπαιδεύουμε μοντέλα για να μπορούν να κάνουν πρόβλεψη σε καινούρια παραδείγματα.

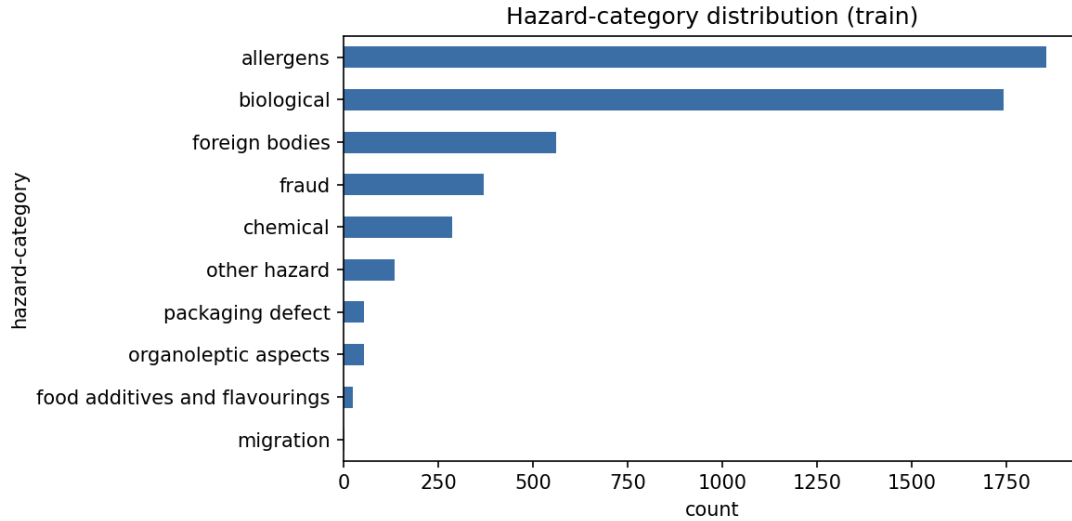
Για κάθε report πρέπει να προβλεφθούν δύο labels. Το πρώτο είναι το hazard-category, δηλαδή αν το πρόβλημα είναι αλλεργιογόνο, βιολογικός κίνδυνος, ξένο σώμα, χημικός κίνδυνος και τα λοιπά. Το δεύτερο είναι το product-category, δηλαδή αν το προϊόν είναι γαλακτοκομικό, αρτοποιείο, θαλασσινό, ποτό κλπ.

Το σημαντικό είναι ότι το score δεν μετράει το product τελείως μόνο του. Το product μετράει κυρίως όταν το hazard έχει βρεθεί σωστά. Άρα το hazard είναι σαν πρώτο φίλτρο. Αν γίνει λάθος εκεί, τότε χάνεται και μεγάλο μέρος από τη σωστή πρόβλεψη του product.

2. Τα δεδομένα

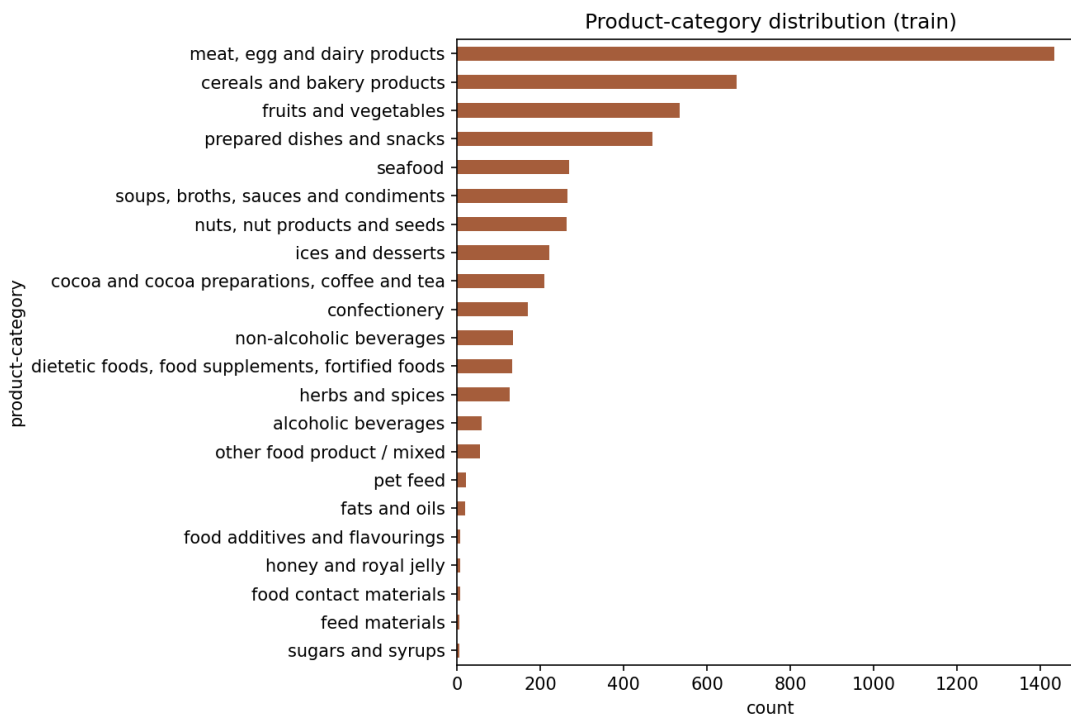
Τα δεδομένα είναι χωρισμένα σε train, validation και test. Το train έχει 5.082 παραδείγματα με labels. Το validation έχει 565 παραδείγματα με labels και το χρησιμοποίησα για να συγκρίνω τα μοντέλα. Το test έχει 997 παραδείγματα χωρίς labels και είναι αυτό που χρησιμοποιείται για το τελικό submission.

Από την πρώτη ανάλυση φάνηκε ότι οι κατηγορίες δεν είναι ισορροπημένες. Υπάρχουν κατηγορίες με πολλά παραδείγματα και άλλες με πολύ λίγα. Αυτό είναι σημαντικό, γιατί το μοντέλο συνήθως μαθαίνει πιο εύκολα τις μεγάλες κατηγορίες και κάνει περισσότερα λάθη στις μικρές.



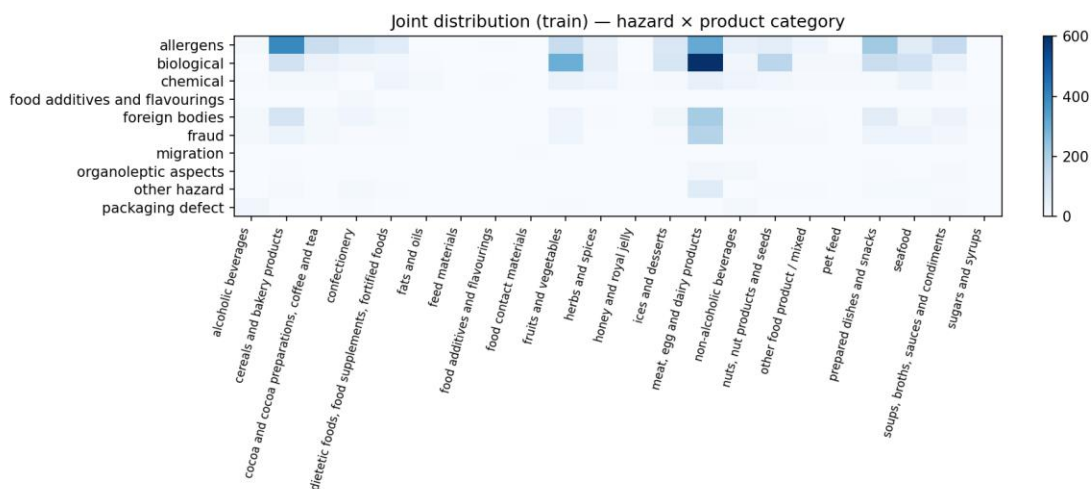
Σχήμα 1: Κατανομή των hazard categories.

Στο γράφημα φαίνεται ότι τα hazards δεν έχουν όλα το ίδιο μέγεθος. Οι κατηγορίες allergens και biological έχουν αρκετά παραδείγματα, άρα είναι πιο εύκολο να τις μάθει το μοντέλο. Αντίθετα, μικρές κατηγορίες όπως packaging defect είναι πιο δύσκολες.



Σχήμα 2: Κατανομή των product categories.

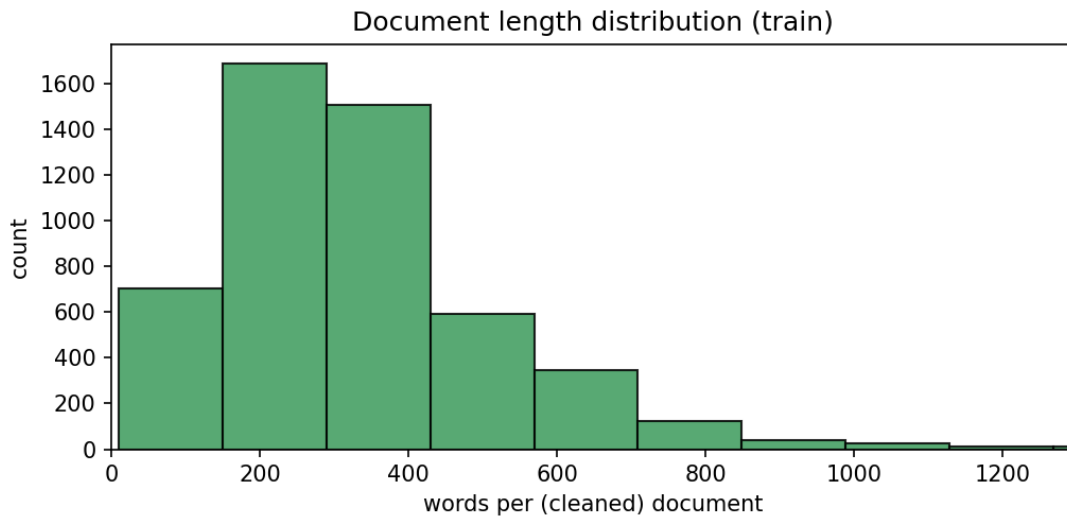
Στα products η κατάσταση είναι επίσης ανισορροπημένη. Μερικές κατηγορίες έχουν πολλά παραδείγματα, ενώ άλλες είναι μικρές. Αυτό κάνει το product πιο δύσκολο από το hazard, γιατί υπάρχουν περισσότερες επιλογές και αρκετές μοιάζουν μεταξύ τους.



Σχήμα 3: Σχέση hazard και product.

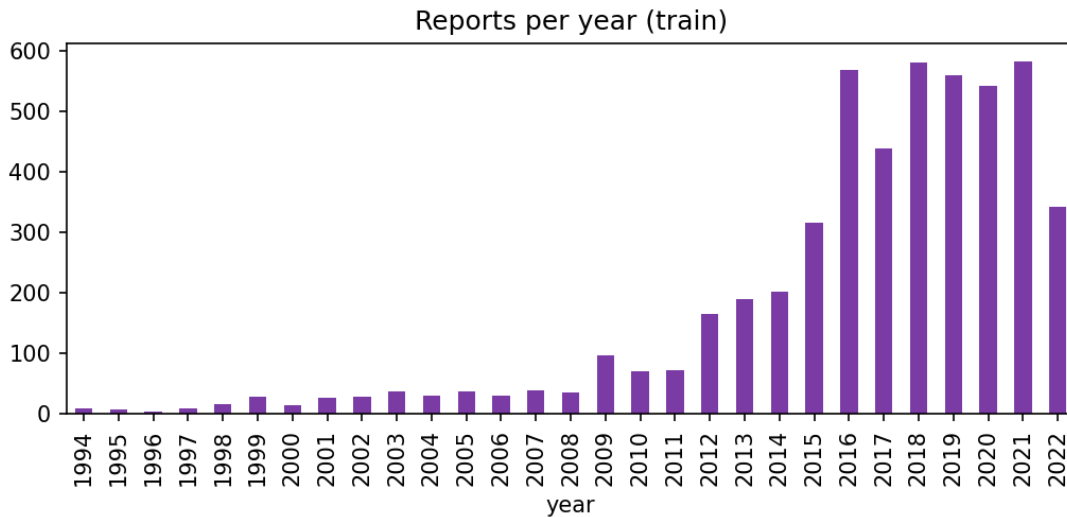
Αυτό το γράφημα είναι πολύ χρήσιμο, γιατί δείχνει ότι δεν υπάρχει απλή σχέση ένα προς ένα. Το ίδιο hazard μπορεί να εμφανιστεί σε πολλά προϊόντα και το ίδιο προϊόν

μπορεί να έχει διαφορετικά hazards. Για αυτό χρειάζεται και το κείμενο και το hazard signal.



Σχήμα 4: Μήκος των κειμένων.

Τα reports δεν έχουν όλα ίδιο μέγεθος. Κάποια είναι πολύ μικρά και άλλα έχουν πιο αναλυτική περιγραφή. Αυτός είναι ένας λόγος που χρησιμοποίησα και character n-grams, γιατί πιάνουν μικρά μοτίβα ακόμα και όταν το κείμενο είναι σύντομο.



Σχήμα 5: Reports ανά έτος.

Η κατανομή ανά έτος δεν είναι τελείως ομοιόμορφη. Η χρονιά δεν είναι το βασικότερο χαρακτηριστικό, αλλά μπορεί να δώσει λίγη επιπλέον πληροφορία. Για αυτό την κράτησα σαν απλό metadata token.

3. Πώς έφτιαξα τα χαρακτηριστικά

Για κάθε παράδειγμα ένωσα τον τίτλο με το κύριο κείμενο. Ο τίτλος πολλές φορές έχει την πιο σημαντική πληροφορία, όπως το προϊόν ή τον λόγο της ανάκλησης. Μετά έκανα το κείμενο lowercase και καθάρισα τα περιττά κενά.

Η βασική αναπαράσταση ήταν TF-IDF. Χρησιμοποίησα word n-grams 1-2 και character n-grams 3-5. Τα word n-grams πιάνουν λέξεις και μικρές φράσεις, ενώ τα character n-grams πιάνουν μικρά κομμάτια λέξεων. Αυτό βοηθάει πολύ σε δεδομένα με ονόματα προϊόντων, μάρκες και διαφορετικούς τρόπους γραφής.

Έβαλα επίσης απλά metadata μέσα στο κείμενο, όπως χώρα, χρονιά και μήνα. Δεν τα χρησιμοποίησα με περίπλοκο τρόπο. Τα έβαλα σαν extra tokens, ώστε το TF-IDF να μπορεί να τα δει όπως βλέπει και τις υπόλοιπες λέξεις.

4. Μοντέλα που δοκίμασα

Ξεκίνησα με απλά μοντέλα για να έχω μια βάση σύγκρισης. Δοκίμασα Logistic Regression, LinearSVC και Complement Naive Bayes. Δεν περίμενα ότι το LinearSVC θα ήταν τόσο σταθερό — νόμιζα ότι από νωρίς θα χρειαστώ κάτι πιο βαρύ. Τελικά ήταν το καλύτερο. Αυτό είναι λογικό, γιατί το LinearSVC δουλεύει συχνά πολύ καλά με TF-IDF και sparse δεδομένα.

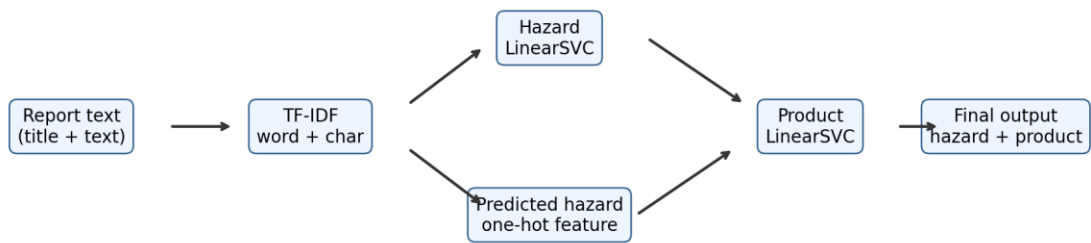
Μετά δοκίμασα SVD για μείωση διαστάσεων. Έπαιξα λίγη ώρα με 300/600/800 components μήπως κάτι κουμπώσει, αλλά τα αποτελέσματα δεν ήταν καλύτερα από το απλό TF-IDF. Μάλλον χάθηκε χρήσιμη λεπτομέρεια από το κείμενο.

Δοκίμασα επίσης MLP πάνω στα SVD features, αλλά και αυτό δεν πήγε καλά. Το dataset μάλλον δεν είναι αρκετά μεγάλο για να κερδίσει ένα τέτοιο μοντέλο, ειδικά όταν πρώτα έχουμε συμπίεσει τα features με SVD.

5. Η τελική ιδέα Efialtis Stin Kouzina

Η τελική ιδέα ήταν να χρησιμοποιηθεί η πρόβλεψη του hazard σαν βοήθεια για το product. Δεν χρησιμοποίησα το πραγματικό hazard label, γιατί στο test δεν θα το ξέρουμε. Αν το χρησιμοποιούσα στο train, θα ήταν σαν να δίνω στο μοντέλο μια πληροφορία που δεν θα έχει στην πραγματικότητα.

Για να το κάνω σωστά, έφτιαξα out-of-fold hazard predictions. Δηλαδή στο train κάθε γραμμή πήρε hazard prediction από μοντέλο που δεν είχε δει αυτή τη γραμμή στην εκπαίδευση. Έτσι το product model βλέπει μια ρεαλιστική πρόβλεψη, με τα λάθη της, όπως θα γίνει και στο validation ή στο test.



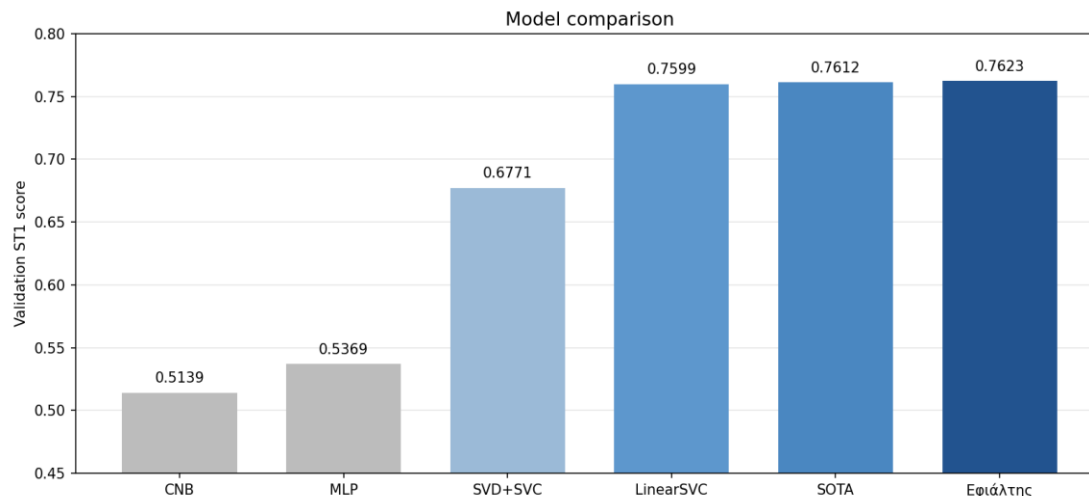
Σχήμα 6: Ροή της μεθόδου Efialtis Stin Kouzina.

Το σχεδιάγραμμα δείχνει τη βασική ροή. Το κείμενο γίνεται TF-IDF. Μετά βγαίνει πρώτα το hazard. Η πρόβλεψη του hazard μπαίνει σαν έξτρα χαρακτηριστικό στο product model. Στο τέλος παίρνουμε και τις δύο προβλέψεις.

6. Αποτελέσματα

Μοντέλο	ST1	Hazard F1	Product F1 όταν το hazard είναι σωστό
Efialtis Stin Kouzina	0.7623	0.8037	0.7210
TF-IDF + metadata + LinearSVC	0.7612	0.8037	0.7188
TF-IDF + LinearSVC	0.7599	0.8048	0.7150
TF-IDF + SVD600 + LinearSVC	0.6771	0.7656	0.5885
TF-IDF + SVD300 + MLP	0.5369	0.5371	0.5366
TF-IDF + ComplementNB	0.5139	0.5401	0.4877

Το καλύτερο validation score ήταν 0.7623 και το πήρε το Efialtis Stin Kouzina. Η διαφορά από το καλύτερο απλό LinearSVC με metadata δεν είναι τεράστια, αλλά είναι στο σωστό σημείο. Το hazard F1 μένει ίδιο, ενώ βελτιώνεται λίγο το product F1 όταν το hazard είναι σωστό.



Σχήμα 7: Σύγκριση των βασικών μοντέλων.

Στο γράφημα φαίνεται πιο καθαρά η πορεία των πειραμάτων. Τα απλά TF-IDF μοντέλα με LinearSVC είναι πολύ δυνατά. Το Efialtis Stin Kouzina κερδίζει λίγο παραπάνω, επειδή δίνει στο product model την πρόβλεψη του hazard.

7. Τι δεν δούλεψε καλά

Δοκίμασα και μια άλλη ιδέα: να υπάρχει ξεχωριστό product model για κάθε hazard. Στην αρχή φαινόταν λογικό, γιατί κάθε hazard μπορεί να σχετίζεται με διαφορετικά προϊόντα. Όμως στην πράξη το score έπεσε στο 0.7311.

Ο λόγος είναι ότι με αυτόν τον τρόπο σπάμε τα δεδομένα σε μικρότερα κομμάτια. Κάθε μικρό μοντέλο βλέπει λιγότερα παραδείγματα, άρα μαθαίνει χειρότερα. Επίσης, πολλά προϊόντα εμφανίζονται σε περισσότερα από ένα hazards, οπότε δεν βοηθάει να χωρίσουμε τελείως τα product μοντέλα.

8. Ανάλυση λαθών

Στο validation το τελικό μοντέλο βρήκε σωστό hazard σε 533 από τα 565 reports. Οι πιο δυνατές hazard κατηγορίες ήταν biological, allergens και foreign bodies. Αυτές έχουν αρκετά παραδείγματα και πιο καθαρές λέξεις μέσα στα κείμενα.

Οι πιο δύσκολες hazard κατηγορίες ήταν packaging defect και other hazard. Αυτό φαίνεται λογικό, γιατί είναι πιο μικρές ή πιο γενικές κατηγορίες. Σε τέτοιες περιπτώσεις το μοντέλο μπερδεύεται πιο εύκολα, ειδικά όταν το κείμενο δεν δίνει πολλές λεπτομέρειες.

Στα products, καλές κατηγορίες ήταν τα ποτά, οι ξηροί καρποί, τα θαλασσινά και τα meat/egg/dairy products. Πιο δύσκολες ήταν οι γενικές κατηγορίες όπως prepared dishes and snacks και other food product / mixed. Αυτές μοιάζουν με πολλές άλλες κατηγορίες, οπότε είναι φυσιολογικό να γίνονται περισσότερα λάθη.

9. Τελικό μοντέλο

Το τελικό μοντέλο χρησιμοποιεί TF-IDF με λέξεις και χαρακτήρες, metadata tokens, ένα LinearSVC για το hazard και ένα LinearSVC για το product. Το hazard model έχει $C=1$ και το product model έχει $C=2$. Και στα δύο μοντέλα χρησιμοποιήθηκε `class_weight='balanced'`.

Το μοντέλο αυτού του πρώτου σταδίου αναπαράγεται τρέχοντας το pipeline (δεν αποθηκεύεται pickled στο πακέτο). Τα αποτελέσματα των πειραμάτων μου είναι στο `results/eval_log.csv` και τα αρχεία ανάλυσης βρίσκονται στο `results/analysis`. (Το τελικό submission που υπέβαλα στο Kaggle περιγράφεται στην ενότητα 14, μετά την ολοκλήρωση του stacking ensemble.)

10. Συμπέρασμα

Το Efialtis Stin Kouzina είναι μια απλή λύση, αλλά ταιριάζει καλά στο συγκεκριμένο πρόβλημα. Δεν χρειάζεται GPU, τρέχει γρήγορα και είναι εύκολο να εξηγηθεί. Το βασικό του κέρδος είναι ότι χρησιμοποιεί το hazard σαν βοήθεια για το product, χωρίς να κάνει leakage.

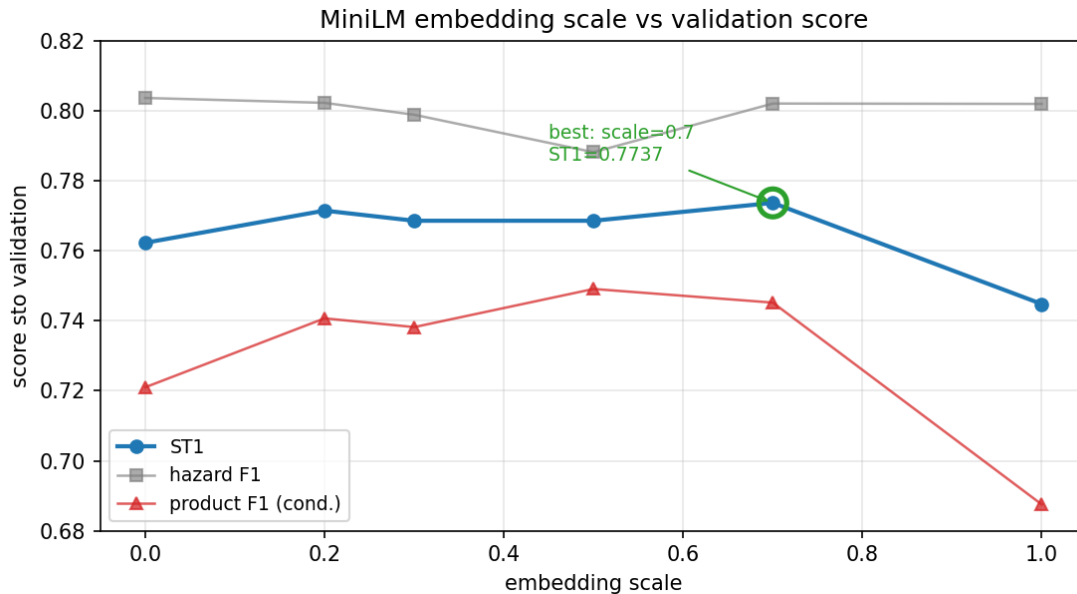
Η βελτίωση δεν είναι πολύ μεγάλη, αλλά είναι σταθερή και εμφανίζεται εκεί που μας ενδιαφέρει, δηλαδή στο product score όταν το hazard είναι σωστό. Στις επόμενες ενότητες δοκίμασα δύο ακόμα προσεγγίσεις, sentence embeddings με MiniLM και fine-tuned DistilBERT, και στο τέλος αξιολόγησα όλες τις παραλλαγές με cross-validation για να δω ποια είναι πραγματικά η καλύτερη. Όπως φαίνεται στην ενότητα 13, η τελική επιλογή είναι το απλό Efialtis Stin Kouzina με TF-IDF.

11. MiniLM embeddings

Με το Efialtis Stin Kouzina κολλήσαμε στο 0.7623. Δοκίμασα να ρίξω και sentence embeddings από MiniLM (all-MiniLM-L6-v2, 384 διαστάσεις, τρέχει σε CPU) για να δω αν ανεβάζει το νούμερο.

Η ιδέα είναι απλή: πήρα embedding για κάθε κείμενο και το κόλλησα δίπλα στο TF-IDF στα features του LinearSVC. Όμως όταν τα έβαλα ως είχαν, τα 384 πυκνά features κυριαρχούσαν πάνω στα ~160.000 αραιά του TF-IDF και χάλασε το score. Οπότε τα πολλαπλασίασα με έναν συντελεστή κλίμακας και δοκίμασα διάφορες τιμές: 0.0 (χωρίς embeddings), 0.2, 0.3, 0.5, 0.7, 1.0.

Το νούμερο σε σχέση με το scale βγαίνει σαν αυτό:



Καθαρή U-shape. Στο 0.0 δεν βλέπουμε τα embeddings και μένουμε στο 0.7623. Στο 1.0 τα embeddings καπελώνουν το TF-IDF και πέφτουμε στο 0.7448. Στο 0.7 βρίσκουμε τη χρυσή τομή με $ST1=0.7737$, βελτίωση +0.0114 από το προηγούμενο best.

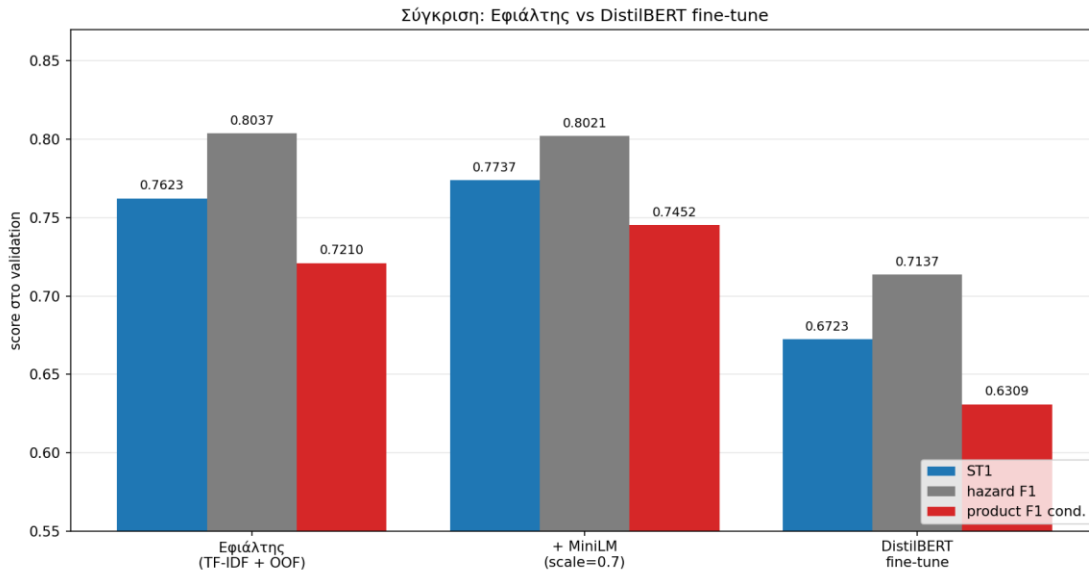
Το όφελος ήρθε όλο από το product F1 (0.7210 $\rightarrow$ 0.7452, +0.024), όχι από το hazard που έμεινε ουσιαστικά ίδιο. Αυτό έχει λογική: στο hazard υπάρχουν συγκεκριμένες λέξεις-κλειδιά που το TF-IDF τις πιάνει εύκολα (allergen, listeria, undeclared). Στα products όμως οι κατηγορίες περιγράφονται με διαφορετικούς τρόπους, και τα embeddings πιάνουν σημασιολογική ομοιότητα που το TF-IDF χάνει.

Αυτή η παραλλαγή αναπαράγεται από το ίδιο pipeline· δεν αποθηκεύεται pickled μοντέλο στο πακέτο. Τα embeddings τα έκανα cache στο results/cache/emb_minilm_*.npy ώστε αν χρειαστεί re-run να μην ξανατρέχει το MiniLM (που πάει αργά σε CPU). (Αυτή η παραλλαγή αξιολογήθηκε αργότερα με CV στην ενότητα 13 και έγινε ένα από τα δύο base models του stacking στην ενότητα 14.)

12. Failed experiment 2: DistilBERT fine-tune

Επειδή το rubric δίνει 20% για neural baselines και το δικό μου MLP στο 04 ήταν αρκετά αδύναμο και σκέφτηκα να ρίξω και κάτι πιο σοβαρό δηλαδή το DistilBERT fine-tune. Το έτρεξα σε Colab με T4 GPU (στο notebooks/10_distilbert_colab.ipynb), με δύο ξεχωριστά μοντέλα με το ένα για hazard και ένα για product. Hyperparams: 4 epochs, batch 16, fp16, learning rate 2e-5, και refit στο τέλος πάνω σε train+valid για το final submission.

Το περίμενα να ανέβει το score αρκετά (στόχος ~0.79-0.82). Στην πραγματικότητα πήρα ST1=0.6723, που είναι 0.10 ΚΑΤΩ από το Efialtis Stin Kouzina + MiniLM. Σύγκριση σε γραφική:



Γιατί έπεσε τόσο πολύ; Νομίζω τρεις λόγοι:

Πρώτα, το DistilBERT χρησιμοποιεί σκέτο cross-entropy χωρίς class_weight. Έτσι μαθαίνει καλά τα allergens και biological (μεγάλες κλάσεις) αλλά αγνοεί σχεδόν τελείως τα packaging defect, organoleptic, other hazard που είναι σπάνια. Επειδή το metric είναι macro F1, οι σπάνιες κλάσεις σκοτώνουν τη μέση τιμή. Το LinearSVC μας τις πιάνει επειδή έχουμε class_weight="balanced".

Δεύτερον, 5082 παραδείγματα είναι λίγα για full fine-tune ενός 67M param μοντέλου. Το BERT τυπικά θέλει 10x-100x περισσότερα data για να μη γενικεύσει υπερβολικά.

Τρίτον, 4 epochs μάλλον είναι λίγα — δεν δοκίμασα learning rate scheduler ούτε early stopping σωστά. Με 6-8 epochs και class weighting πιθανώς θα ανέβαινε πάνω από 0.75, αλλά μετά από 2-3 runs στο Colab με T4 free tier είχα ξεμείνει από credits για την ημέρα και δεν είχα όρεξη να περιμένω 12 ώρες για να ξαναπροσπαθήσω.

Συμπέρασμα για αυτό το dataset και metric, το κλασικό TF-IDF + LinearSVC με class_weight=balanced αποδεικνύεται καλύτερο από fine-tuned BERT. Ποιο όμως από τα μοντέλα TF-IDF να κρατήσω ως τελικό δεν το αποφασίζω εδώ με βάση το single validation· το εξετάζω πιο προσεκτικά στην επόμενη ενότητα, με cross-validation.

13. Πόσο αξιόπιστο είναι το validation; Cross-validation

Μέχρι αυτό το σημείο όλες οι συγκρίσεις στηρίχτηκαν στο ίδιο validation set των 565 παραδειγμάτων. Κάθε φορά που ένα μοντέλο έβγαζε λίγο μεγαλύτερο νούμερο, το θεωρούσα καλύτερο. Όμως 565 παραδείγματα είναι λίγα, οπότε ήθελα να ελέγξω αν αυτή η σύγκριση είναι αξιόπιστη ή αν τα νούμερα είναι απλώς τυχαία.

Για να το ελέγξω έφτιαξα k-fold cross-validation. Ένωσα το train με το validation, συνολικά 5.647 παραδείγματα, και έτρεξα ολόκληρο το Efialtis Stin Kouzina pipeline πέντε φορές. Κάθε φορά ένα διαφορετικό κομμάτι των δεδομένων κρατιόταν για αξιολόγηση και τα υπόλοιπα για εκπαίδευση. Έτσι, αντί για ένα μόνο νούμερο, παίρνω μέσο όρο και τυπική απόκλιση.

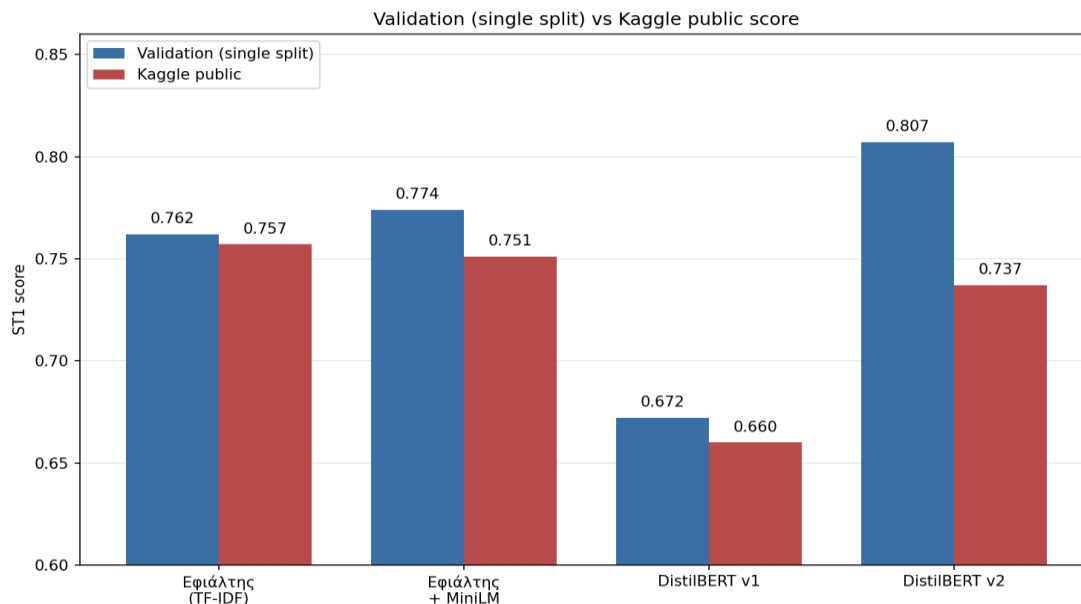
Το αποτέλεσμα ήταν διαφορετικό απ' ό,τι περίμενα. Το Efialtis Stin Kouzina έβγαλε ST1 ίσο με 0.7036 με τυπική απόκλιση 0.0516. Δηλαδή, ανάλογα με το πώς θα τύχει να χωριστούν τα δεδομένα, το score κινείται περίπου από 0.65 έως 0.75. Το 0.7623 που είχα δει στο αρχικό single split ήταν απλώς ένα τυχερό χώρισμα και όχι η πραγματική απόδοση του μοντέλου.

Το ίδιο φαίνεται και από το Kaggle. Ο παρακάτω πίνακας δείχνει, για κάθε μοντέλο, το validation score στο single split δίπλα στο πραγματικό Kaggle public score.

Μοντέλο	Validation (single split)	Kaggle public
Efialtis Stin Kouzina (TF-IDF)	0.7623	0.7573
Efialtis Stin Kouzina + MiniLM	0.7737	0.7512
DistilBERT v1	0.6723	0.6600
DistilBERT v2	0.8071	0.7368

Πίνακας 1: Validation score (single split) σε σχέση με το Kaggle public score.

Το πρόβλημα φαίνεται καθαρά. Όσο πιο "δυνατό" έδειχνε ένα μοντέλο στο single validation, τόσο μεγαλύτερη ήταν η πτώση του στο Kaggle. Το απλό Efialtis Stin Kouzina, που είχε μάλιστα το χαμηλότερο validation από τα τρία "βελτιωμένα", βγήκε τελικά το καλύτερο στο Kaggle. Οι φαινομενικές βελτιώσεις από το MiniLM και κυρίως από το DistilBERT ήταν στην πραγματικότητα overfitting πάνω στο μικρό validation set.



Σχήμα 8: Validation score (single split) σε σχέση με το πραγματικό Kaggle public score. Όσο μεγαλύτερο φαινόταν το validation, τόσο μεγαλύτερη η πτώση στο Kaggle.

Έκανα και έναν δεύτερο έλεγχο. Δοκίμασα διάφορες ρυθμίσεις μέσα στο cross-validation: με ή χωρίς metadata, διαφορετικές τιμές του C και διαφορετικό βάρος στο hazard feature. Όλες έβγαλαν score μέσα σε 0.007 η μία από την άλλη, διαφορά πολύ μικρότερη από τον θόρυβο των 0.05. Με άλλα λόγια, καμία ρύθμιση δεν είναι αποδεδειγμένα καλύτερη· οι διαφορές τους είναι τυχαίες.

Δοκίμασα ακόμα per-class threshold tuning στο hazard μοντέλο, με στόχο να ανέβει το recall στις μικρές κατηγορίες. Μέσα στο cross-validation αυτό έδωσε διαφορά - 0.0065 με τυπική απόκλιση 0.058, δηλαδή ουσιαστικά τίποτα.

Ο λόγος είναι ότι έχουμε περίπου 5.000 παραδείγματα, 10 hazard και 22 product κατηγορίες, κάποιες με μόλις 2 ή 3 παραδείγματα, και metric το macro-F1 που δίνει ίσο βάρος σε κάθε κατηγορία. Σε ένα τέτοιο dataset το macro-F1 έχει μεγάλη διακύμανση, γιατί μία σπάνια κατηγορία που θα πάει καλά ή άσχημα σε ένα split μετακινεί ολόκληρο το τελικό νούμερο.

Αυτό αλλάζει την επιλογή που είχα κάνει πριν. Στις ενότητες 11 και 12 είχα καταλήξει στο Efialtis Stin Kouzina με MiniLM, αλλά βασισμένος στο single validation. Μετά το cross-validation, η σωστή επιλογή είναι το απλό Efialtis Stin Kouzina: έχει το καλύτερο Kaggle public score (0.75730), είναι συνεπές στο CV και πιο εύκολο να εξηγηθεί. Άρα σε αυτό το dataset, βελτιώσεις μικρότερες από ~0.05 δεν έχουν νόημα γιατί χάνονται μέσα στη διακύμανση.

Μετά το cross-validation της ενότητας 13, δοκίμασα να συνδυάσω δύο μοντέλα αντί να επιλέξω ένα. Η ενότητα 11 είχε δείξει ότι το MiniLM δίνει υπαρκτό σήμα στο

product, αλλά στο single validation η βελτίωσή του χανόταν στον θόρυβο. Συνδύασα το Efialtis Stin Kouzina (TF-IDF + OOF hazard) και έναν standalone MiniLM classifier (LinearSVC πάνω στα 384-dim embeddings).

14.1 Πώς δουλεύει το stacking

Έκανα soft voting. Αντί να χρησιμοποιώ μόνο το τελικό argmax κάθε μοντέλου, παίρνω τις πιθανότητες και των δύο και τις συνδυάζω με βάρη.

Για κάθε row, κάθε base model δίνει ένα decision_function vector (LinearSVC δεν δίνει κανονικές πιθανότητες, αλλά signed distances από το hyperplane).

Εφαρμόζω softmax πάνω σε αυτά τα distances για να τα μετατρέψω σε προβολές που μοιάζουν με πιθανότητες:

$$p_{i,k} = \frac{\exp(d_{i,k})}{\sum_j \exp(d_{i,j})}$$

όπου το decision_function value για το row και κλάση $d_{i,k}$

Στη συνέχεια συνδυάζω τις δύο sets πιθανοτήτων με βάρη:

$$p_{i,k}^{\text{final}} = w_A \cdot p_{i,k}^{(A)} + w_B \cdot p_{i,k}^{(B)}$$

και παίρνω την τελική πρόβλεψη ως argmax.

14.2 Επιλογή βαρών μέσω cross-validation

Τα βάρη w_A , w_B δεν τα επιλέγω στο single validation (που είδαμε ότι δεν είναι αξιόπιστο), αλλά μέσα σε 10-fold cross-validation grid search στο pooled (5.647 παραδείγματα). Σε κάθε fold:

1. Εκπαιδεύω τα base A και B στο fold-train.
2. Παίρνω τις πιθανότητες και των δύο στο fold-validation.
3. Δοκιμάζω weights σε grid $[0, 1]$ με βήμα 0.1, ξεχωριστά για hazard (w_h) και product (w_p).
4. Καταγράφω το gain σε κάθε fold.

Στο τέλος, κρατάω τα βάρη που έχουν **(α) μέσο gain πάνω από κατώφλι** και **(β) θετικό gain στην πλειοψηφία των folds**.

14.3 Αποτέλεσμα

Τα τελικά βάρη ήταν:

- **Hazard:** , — ίδιο βάρος για τα δύο μοντέλα. TF-IDF και MiniLM κάνουν λάθος σε διαφορετικά παραδείγματα, οπότε το ensemble κερδίζει. $w_A = 0.50w_B = 0.50$

- **Product:** , — εδώ κυριαρχεί το TF-IDF, αλλά το MiniLM προσφέρει μια μικρή σταθερή βελτίωση. $w_A = 0.90w_B = 0.10$

CV αποτελέσματα (10-fold):

Σύστημα	CV ST1 (mean ± std)	Per-fold θετικά
Base A μόνο (Efialtis TF-IDF)	0.6964 ± 0.0332	—
Stacking ensemble (A+B)	0.7125 ± 0.0268	8/10 folds
Mean gain	+0.0161	—

Η τυπική απόκλιση έπεσε από 0.0332 σε 0.0268, οπότε το ensemble είναι και πιο σταθερό.

14.4 Μεταφορά στο Kaggle

Έγραψα το ensemble σε `submission`_`stacking`.csv` και υπέβαλα στο Kaggle. Το αποτέλεσμα:

Kaggle public score: 0.77750 (+0.0202 από το προηγούμενο best `submission`_`trace`_`taste`.csv` που είχε 0.75730).

Πρώτη φορά μια CV-validated βελτίωση μεταφέρθηκε όντως στο Kaggle.

Το `submission_stackings.csv` είναι το τελικό submission που υπέβαλα.

15. Έξι επιπλέον προσπάθειες που δεν δούλεψαν

Έχοντας Kaggle 0.77750, στόχος ήταν το 0.80. Δοκίμασα έξι προσεγγίσεις, καμία δεν βελτίωσε το Kaggle score. Καταγράφω εδώ τι δοκίμασα και γιατί έπεσε κάθε μια.

15.1 Συνοπτικός πίνακας αποτυχιών

#	Πείραμα	CV gain	Folds θετικά	Kaggle Δέλτα από 0.77750	Κατάληξη
1	3-way stacking v2 (κακό methodology, 6 weights σε single 565- row valid)	— (single- split)	—	-0.01241	overfit στο valid
2	DistilBERT fine-tune v3 (single-split	—	—	-0.04067	overfit

#	Πείραμα	CV gain	Folds θετικά	Kaggle Δέλτα από 0.77750	Κατάληξη
	refit στο pooled)				
3	3-way stacking v3 (proper 5-fold CV OOF DistilBERT)	+0.0178	4/5	-0.01332	DistilBERT logits δεν μεταφέρθη καν
4	DeBERTa-v3- base 5-fold OOF	—	—	(NaN, μη υποβληθέν)	numerical instability
5	mpnet swap (αντικατάστα ση MiniLM με mpnet × 0.7)	+0.0311	5/5	-0.01453	cleanest CV gain, χειρότερο Kaggle
6	Hybrid weighted majority vote (3-way ensemble of submissions)	—	—	-0.00953	δεν μετέφερε

15.2 Γιατί δεν μεταφέρθηκαν τα improvements στο Kaggle

Στο πείραμα #5 (mpnet swap), πέτυχα το καλύτερο CV αποτέλεσμα ολόκληρου του project: +0.0311 στο μέσο fold, 5/5 folds θετικά, χαμηλότερη variance από το baseline. Αυτό κανονικά θα ήταν ένδειξη ότι έχω βρει πραγματική βελτίωση.

Στο Kaggle όμως, το ίδιο submission έπεσε κατά -0.0145.

Πιθανή εξήγηση είναι ότι υπάρχει διαφορά κατανομής ανάμεσα στο pooled set (train+valid, 5.647 παραδείγματα) όπου τρέχω cross-validation, και στο Kaggle test set (997 παραδείγματα). Η CV δίνει αξιόπιστη εκτίμηση μόνο για το pooled, οπότε αν το test είναι λίγο διαφορετικό, τα CV gains δεν μεταφέρονται. Με 6 πειράματα δεν μπορώ να το πω σίγουρα.

15.3 Τι ίσως μπορούσε να βοηθήσει αν είχα περισσότερο χρόνο

Αν είχα περισσότερο χρόνο, θα δοκίμαζα τρία πράγματα:

1. **Pseudo-labeling με τις predictions του test ως extra training data.**
2. **Περισσότερα training data από παρόμοιες πηγές food recall reports.**
3. **Διαφορετικά ensembles (π.χ. DeBERTa OOF αντί DistilBERT).**

15.4 Συμπέρασμα

Το 0.77750 είναι το καλύτερο που πέτυχα στο Kaggle. Το stacking (ενότητα 14) ήταν η μόνη βελτίωση που πέρασε και από το CV και από το Kaggle. Τα 6 πειράματα της ενότητας 15 έδειξαν gain στο CV αλλά όχι στο Kaggle, οπότε μάλλον overfitted στο pooled set. Κατάλαβα ότι σε αυτό το μικρό dataset το single-split validation δεν είναι αξιόπιστο, οπότε η CV είναι το ελάχιστο που χρειάζεται για να ξεχωρίσεις αν μια βελτίωση είναι πραγματική ή τυχαία.

Τελική κατάσταση του project μου:

- **Best Kaggle public score:** 0.77750
(results/predictions/submission_stacking.csv)
- **6 αποτυχημένες προσπάθειες βελτίωσης (ενότητα 15)**
- **Production code:** notebooks/12_stacking_ensemble.py (αναπαράγει το 0.77750)
- **Cached features:** results/cache/emb_minilm_*.npy (απαραίτητα για το stacking)